

ETAPES SUR LA CREATION D'UNE APPLICATION API REST

1. Introduction

Ce travail pratique avait pour objectif de développer une API REST avec Spring Boot, permettant de gérer des opérations CRUD sur une entité (Product).

Les points clés incluent :

- La création du projet avec Maven
- La mise en place d'un DTO et de la validation
- La persistance des données avec PostgreSQL
- Les tests via Postman
- La gestion des exceptions globales

2. Création du projet

Le projet a été généré Spring Initializr en intégrant les dépendances nécessaires :

- spring-boot-starter-web (Spring Web)
- spring-boot-starter-data-jpa (Spring Data JPA)
- spring-boot-starter-validation (Validation)
- org.postgresql:postgresql (PostgreSQL)
- lombok

3. Structure du projet

La structure adoptée repose sur une séparation claire des responsabilités :

- controller : pour mettre les endpoints ou les URI REST
- service : là où il y a la logique métier
- repository : accès aux données (fonctions pour prendre dans la base de données).
- model : où il y a les entités JPA (Product)
- dto : pour mettre les objets de transfert au lieu de tout de suite mettre Product dans le controller.
- exception : là où l'on gère les exceptions

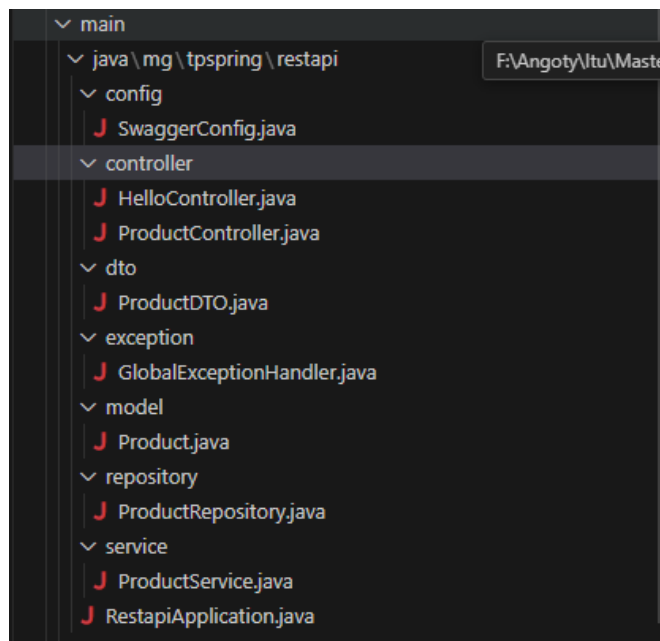


Image 1 : Schéma de l'architecture du projet

4. Implémentation du CRUD

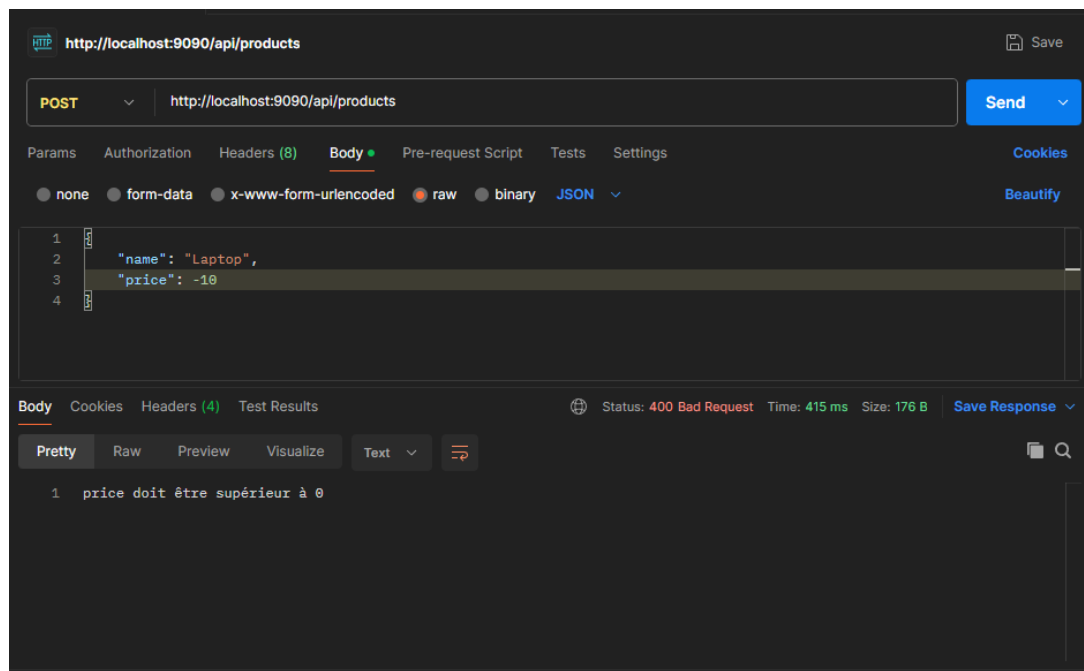
L'entité Product comporte trois attributs principaux : id, name et price. Le repository JPA assure la persistance, tandis que le service expose les méthodes findAll, findById, et save. Le controller fournit les endpoints /api/products pour les opérations CRUD classique : lecture de tous les produits, lecture par identifiant, création et suppression.

5. DTO et validation

Un ProductDTO a été introduit afin de séparer la couche de la présentation de la persistance. Les annotations `@NotBlank` et `@Positive` garantissent la validité des données. L'annotation `@Valid` dans le controller déclenche la validation automatique. En effet, les erreurs sont gérées par `GlobalExceptionHandler`, qui renvoie une réponse 400 pour les erreurs de validation et 500 pour les exceptions génériques.

```
src > main > java > mg > tpspring > restapi > exception > GlobalExceptionHandler.java > GlobalExceptionHandler > handleValidationEr
1  package mg.tpspring.restapi.exception;
2
3  import org.springframework.http.ResponseEntity;
4  import org.springframework.web.bind.MethodArgumentNotValidException;
5  import org.springframework.web.bind.annotation.ExceptionHandler;
6  import org.springframework.web.bind.annotation.RestControllerAdvice;
7
8  @RestControllerAdvice
9  public class GlobalExceptionHandler {
10
11     // Gestion des erreurs de validation
12     @ExceptionHandler(MethodArgumentNotValidException.class)
13     public ResponseEntity<String> handleValidationError(MethodArgumentNotValidException e) {
14         String errorMessage = e.getBindingResult().getFieldErrors()
15             .stream()
16             .map(err -> err.getField() + " " + err.getDefaultMessage())
17             .findFirst()
18             .orElse(other: "Validation error");
19         return ResponseEntity.badRequest().body(errorMessage);
20     }
21
22     // Gestion générique des autres erreurs
23     @ExceptionHandler(Exception.class)
24     public ResponseEntity<String> handleError(Exception e) {
25         return ResponseEntity.status(500).body("Server Error");
26     }
27 }
28
```

Img.2 : Gestion des exceptions



Img.3 : Test des données sur postman

6. Base de données.

Après une première utilisation de H2, le projet a migré vers PostgreSQL. La base restapi a été créée et configurée dans `application.properties`. L'option `spring.jpa.hibernate.ddl-auto=update` a permis la génération automatique des tables.

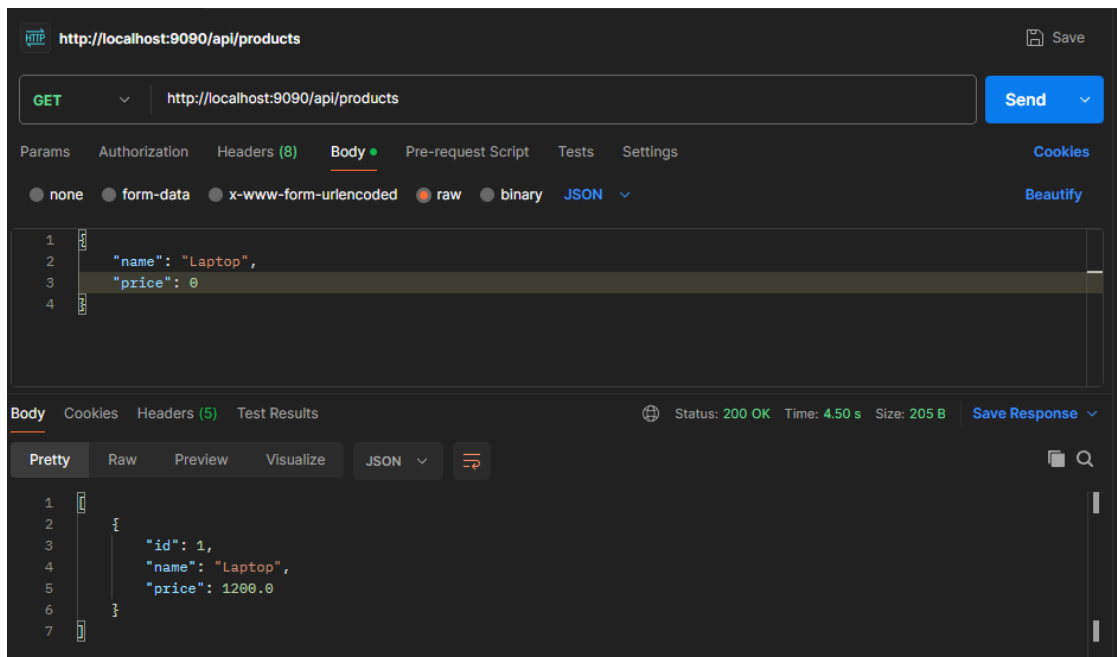
```
restapi=# select * from product;
 id | name | price 
----+-----+-----
  1 | Laptop | 1200 
(1 ligne)
```

Img.4 : Donnés insérées dans la base de données

7. Tests

Les tests réalisés avec Postman ont confirmé le bon fonctionnement de l'API.

- Requête valide (POST /api/products) → création du produit avec code 201.
- Requête invalide (POST /api/products) → erreur 400.
- GET /api/products → liste des produits.
- GET /api/products/{id} → produit par ID.
- PUT /api/products/{id} → mise à jour.
- DELETE /api/products/{id} → suppression.



Img.5 : Test du CRUD sur postman

8. Conclusion

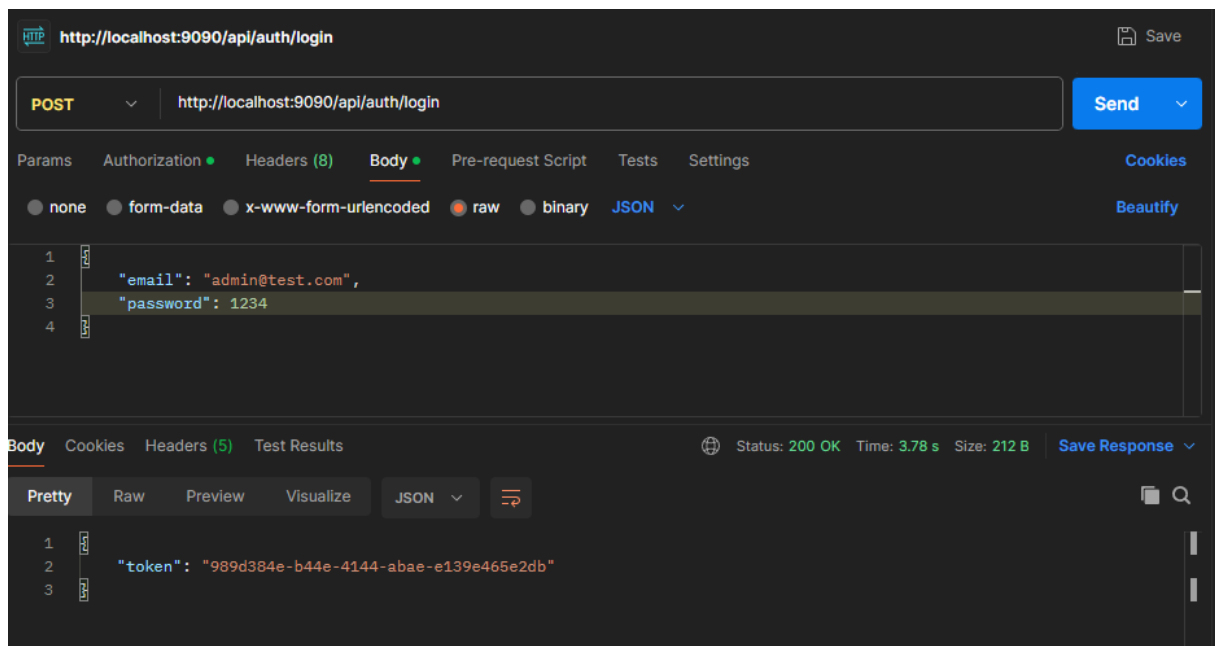
Ce TP a permis de mettre en œuvre une API REST complète avec Spring Boot. Les principales compétences acquises concernent la structuration d'un projet en couches claires (controller, service, repository), l'utilisation des DTOs et de la validation, la configuration d'une base PostgreSQL avec JPA, ainsi que la mise en place de tests pratiques via Postman. La gestion des exceptions et la séparation des responsabilités illustrent les bonnes pratiques de développement d'API modernes.

APPLICATION DE LA GESTION DES TOKENS DANS UNE APPLICATION REST API

On utilise les tokens pour éviter l'accès non sécurisé aux données de l'application.

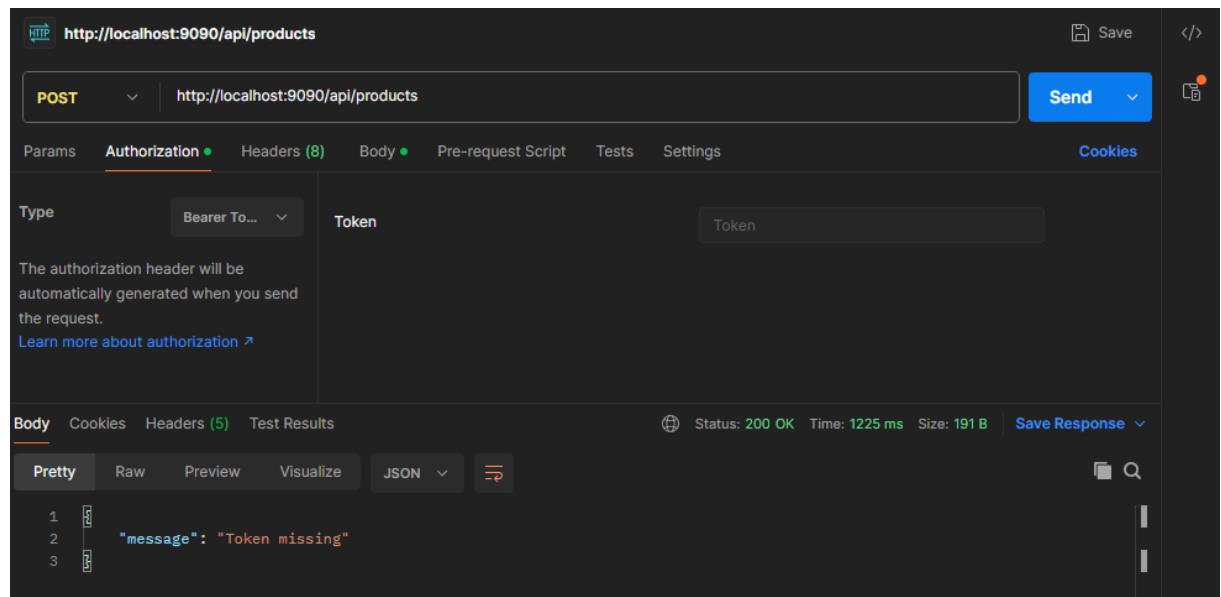
Par exemple, ici on va voir l'application de token dans l'insertion de produits :

Tout d'abord, l'utilisateur se connecte et l'application va de suite, créer un token qui est stockée en mémoire.



Img.6 : Authentification et génération du token

Ensuite l'utilisateur va créer un produit sans le token et on voit bien le message d'erreur qui dit : **token missing**



Img.7 : Création de produits sans token

En effet, la création des produits n'est pas autorisée si l'utilisateur ne fournit pas de token.

Pour être claire dans cette étape de sécurisation avec token, nous allons répondre aux questions suivantes :

- Pourquoi utilise-t-on un token ?

On utilise un token pour éviter d'envoyer un email + password à chaque requête, pour sécuriser les échanges.

- Pourquoi ne pas envoyer un email + password à chaque requête ?

Parce que ce serait lourd et peu sécurisé (les mots de passe circulent en clair).

- Où est stocké le token dans ce TP ?

Dans ce TP, le token est stocké dans le **Set** en mémoire côté serveur.

- Que se passe-t-il si le serveur redémarre ?

Si le serveur redémarre, les tokens sont perdus, il faut se reconnecter pour en générer un nouveau.